

🛠 Documentación Técnica del Sistema Bancario

****Asignaturas:**** Refactoring y Testing | Integración y Entrega Continua

****Año:**** 2026

🏠 1. Arquitectura General y Principios de Diseño

El sistema se estructura bajo una ****arquitectura en capas desacoplada****, diseñada para garantizar alta cohesión, bajo acoplamiento y testabilidad:

[Interfaz Flet (FE)] —> [Lógica de Negocio / Servicios] —> [DAO (SQLAlchemy)] —> [SQLite]

Principios SOLID y Calidad de Software Aplicados

- **Single Responsibility Principle (SRP):** Desacoplamiento estricto entre los Modelos de Persistencia (src/models.py), los Objetos de Dominio (Cliente, Cuenta, Tarjeta), el DAO (src/dao.py) y los Servicios/Capas de Negocio (src/services.py). Cada clase tiene un único propósito delimitado.
- **Don't Repeat Yourself (DRY):** La clase base Persona centraliza las validaciones de formato (nombre, apellido, DNI numérico) y la lógica de hashing de contraseñas. Tanto Cliente como Empleado heredan y reutilizan esta lógica sin duplicar código.
- **Open/Closed Principle (OCP):** La estructura permite extender nuevos tipos de usuarios o roles (ej. nuevos subtipos de Persona) sin modificar la lógica existente en la clase base.
- **Liskov Substitution Principle (LSP):** Cliente y Empleado actúan como subtipos válidos de Persona. Asimismo, CuentaAhorro extiende de Cuenta respetando estrictamente los contratos y firmas de la clase base sin alterar su comportamiento.
- **Inversión de Dependencias (DIP) y Bajo Acoplamiento:** El acceso a datos se abstrae mediante la interfaz del DAO, permitiendo alternar la base de datos de producción (sistema_bancario.db) con una base de datos en memoria (:memory:) para la ejecución automatizada de pruebas sin persistencia destructiva.

- **Encapsulamiento y Validación Temprana:** Los atributos principales se mantienen protegidos (uso de dunder/privados) con acceso controlado mediante getters. Las validaciones ocurridas en el dominio (DNI numérico, montos positivos, saldo suficiente) evitan errores en runtime antes de golpear la base de datos.

2. Capa de Persistencia y Modelo Entidad-Relación (Backend & BD)

Diagrama ER (Mermaid)

Fragmento de código

erDiagram

```
persona {
    INT id_persona PK
    TEXT nombre "NN"
    TEXT apellido "NN"
    TEXT dni "NN"
    TEXT hashedPassword "NN"
    TEXT email "NN"
    TEXT telefono "NN"
    TEXT direccion "NN"
    TIMESTAMP fecha_registro "NN"
}
```

```
cliente {
    INT id_cliente PK
    INT id_persona FK "NN"
    INT categoria
    TEXT estado "NN"
}
```

```
empleado {
```

```
INT id_empleado PK
INT id_persona FK "NN"
TEXT legajo "NN"
TEXT cargo "NN"
TEXT departamento "NN"
TIMESTAMP fecha_ingreso "NN"
REAL salario "NN"
TEXT sucursal "NN"
}
```

```
cuenta {
  INT id_cuenta PK
  TEXT nro_cuenta "NN"
  INT id_cliente FK "NN"
  INT id_tipo_cuenta FK "NN"
  REAL saldo "NN"
  REAL tasa_interes
  TIMESTAMP fecha_creacion "NN"
}
```

```
tipo_cuenta {
  INT id_tipo_cuenta PK
  TEXT nombre "NN"
  TEXT descripcion
}
```

```
transaccion {
  INT id_transaccion PK
```

```

    INT id_cuenta FK "NN"

    INT id_tipo_transaccion FK "NN"

    REAL monto "NN"

    TIMESTAMP fecha "NN"
}

```

```

tipo_transaccion {
    INT id_tipo_transaccion PK

    TEXT nombre "NN"

    TEXT descripcion
}

```

```

persona ||--|| cliente : "id_persona referencia id_persona"
persona ||--|| empleado : "id_persona referencia id_persona"
cliente ||--o{ cuenta : "id_cliente referencia id_cliente"
tipo_cuenta ||--o{ cuenta : "id_tipo_cuenta referencia id_tipo_cuenta"
cuenta ||--o{ transaccion : "id_cuenta referencia id_cuenta"

tipo_transaccion ||--o{ transaccion : "id_tipo_transaccion referencia
id_tipo_transaccion"

```

Mapeo ORM y DAO (src/dao.py & src/models.py)

El módulo **DAO** utiliza SQLAlchemy ORM con SQLite.

- **Control de Autenticación Segura:** Las contraseñas se procesan mediante hashing en Persona utilizando la librería passlib[bcrypt] y se validan con dao.validar_login_por_dni(), asegurando que nunca se guarden ni comparen credenciales en texto plano.
- **Manejo de Sesión:** Autenticacion(dao).autenticar(dni, clave) valida el usuario y retorna un diccionario con el rol ("cliente" o "personal") y sus cuentas reconstruidas desde la base de datos.
- **Integridad Referencial Estricta:** Habilitación explícita de PRAGMA foreign_keys=ON al conectarse a SQLite para validar constraints de clave foránea entre cuenta, tipo_cuenta, transaccion y tipo_transaccion.

⚙️ 3. Módulo de Lógica de Negocio y Servicios (src/services.py)

Encapsula las reglas del negocio entre la presentación y la persistencia:

Alta de Cliente (BE-07)

- **Validación de Unicidad:** Verifica mediante `dao.obtener_cliente_por_dni(dni)` la existencia previa del documento. Si ya existe, lanza un `ValueError("Ya existe un cliente con ese DNI.")`.
- **Creación de Entidad:** Instancia `Cliente(nombre, apellido, dni)` permitiendo que la propia clase aplique sus validaciones de formato de texto (`.strip()`, `.title()`, solo letras) y DNI numérico.
- **Persistencia:** Guarda la entidad a través del DAO y retorna el `id_cliente` generado.

Alta de Cuenta (BE-08 / BE-09)

- **Asociación de Cliente:** Comprueba la existencia activa del cliente previo a la asignación de una nueva cuenta.
- **Validación de Saldo Inicial:** Controla que el saldo inicial sea un valor válido antes de enviar el registro al DAO.

🧪 4. Suite de Pruebas Automatizadas e Integración Continua (CI/CD)

Pruebas Unitarias (tests/test_*.py)

- Estrategia **Arrange-Act-Assert (AAA)** aplicada a las clases de dominio: Cliente, Cuenta, CuentaAhorro, Transaccion y Tarjeta.
- Uso de `@pytest.fixture` para aislar el estado de los objetos.
- Captura de excepciones personalizadas como `SaldoInsuficienteError` y `LimiteExcedidoError`.

Pruebas de Integración (tests/test_dao.py, tests/test_propuesta.py)

- **Persistencia Isolada:** Uso de `DAO(":memory:")` para ejecutar la suite de pruebas sin alterar la base de datos real `sistema_bancario.db`.
- **Sincronización de Saldo y Transacciones:** Validación de operaciones `UPDATE` mediante `dao.actualizar_saldo_cuenta()` y registro de historial en `transaccion` tras cada depósito o retiro.
- **Captura de Restricciones BD:** Control de excepciones de base de datos como `sqlalchemy.exc.IntegrityError` y `ValueError` ante intentos de inserción con claves foráneas inexistentes o DNI duplicados.

Pipeline de CI/CD (GitHub Actions)

Configurado en `.github/workflows/ci.yml`:

- Se dispara automáticamente en eventos push y en aperturas/actualizaciones de Pull Request.
- Despliega un entorno Python 3.10 en contenedores Ubuntu, instala dependencias y ejecuta pytest.
- Bloquea merges a main si alguna prueba de dominio o integración falla.